Perhaps you're wondering if we could do this with a Docker container… of course, we can! We can build our own image from the official PHP 8.4 image with the following Dockerfile…

Docker

```
FROM php:8.4-cli

RUN pear update-channels \
    && pear install PHP_CodeSniffer

ENTRYPOINT ["phpcs"]
CMD ["--version"]
```

This Dockerfile is making use of a feature called `ENTRYPOINT` and overwriting the `CMD` of the base `php:8.4-cli` image. The `ENTRYPOINT` instruction simply sets the container's primary process. In our case, we want the container environment of `php:8.4-cli` to provide everything we need (including the PHP binary itself!) to support the `phpcs` process. Essentially, running `docker run -it ...` with this image `ENTRYPOINT` turns the `docker run ...` command (on the host) into a direct invocation of `phpcs` (inside the container). Now build the image (run this command, making sure the current directory contains the Dockerfile above):

```
$ docker build -t local/php-codesniffer:8.4 .
```

Using Codesniffer

Next, let's create some code that violates some PSR standards. Save the following as `phpcs_broken.php`

```
$num = 1+ 1 ;

class badClassName
{
    function BadMethodName  ( $arg1 , $arg2 ) {}
}
```